

---

# Decision Trees

## BSAD 8310: Business Forecasting — Lecture 4

---

Department of Economics  
University of Nebraska at Omaha  
August 24, 2026

- 
- 1 Why Go Beyond Linear Models?
  - 2 How Decision Trees Work
  - 3 Implementing Decision Trees in Python
  - 4 Limitations and Preview: Ensembles

---

## Why Go Beyond Linear Models?

Real-world relationships are often nonlinear, involve interactions, and shift with context.

---

## Problems with linear models:

- **No interactions:** the holiday-season advertising effect cannot be captured by a constant slope
- **No threshold effects:** spending may be flat until unemployment drops below 5%, then jump
- **Symmetric predictions:** extreme values distort coefficients in both directions equally

Monthly sales depend on:

- Last year's same-month sales (trend)
- Whether it's December (seasonality)
- Unemployment rate (conditions)

The December effect is *larger* in a strong economy — an interaction linear models miss.

Decision trees capture nonlinear relationships and interactions **automatically**, without needing to specify them.

$$\text{Expected MSE} = \underbrace{\text{Bias}^2}_{\text{systematic error}} + \underbrace{\text{Variance}}_{\text{sensitivity to data}} + \underbrace{\sigma^2}_{\text{irreducible noise}}$$

### High bias (underfitting):

- Model is too simple
- Misses real patterns in the data
- Bad on both training AND test sets
- Example: linear model on a curved relationship

### High variance (overfitting):

- Model is too complex
- Memorizes training data noise
- Great on training, bad on test set
- Example: a very deep decision tree

**Goal:** find the right complexity that balances bias and variance — always evaluate *out-of-sample*.

---

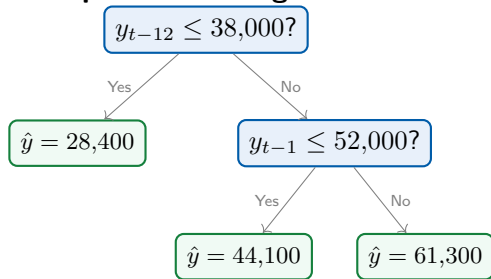
## How Decision Trees Work

Partition the feature space into rectangles; predict the mean in each region.

---

A decision tree works like a flowchart: ask a series of yes/no questions about the input features to arrive at a prediction.

### Example: forecasting retail sales



### Key properties:

- **Non-parametric** — no functional form assumed
- **Nonlinear** — captures threshold effects
- **Interpretable** — decision path is readable
- **Mixed features** — lags, rolling stats, dummies
- **Scale-invariant** — no standardization needed

A single *deep* tree **overfits severely**. Ensembles (Random Forests, XGBoost) fix this.

**CART (Classification and Regression Trees)** builds the tree greedily from top to bottom:  
**At each node, find the best split  $(j, t)$ :**

$$\min_{j, t} \left[ \sum_{i: x_{ij} \leq t} (y_i - \bar{y}_L)^2 + \sum_{i: x_{ij} > t} (y_i - \bar{y}_R)^2 \right]$$

Choose the feature  $j$  and threshold  $t$  that most reduces the within-region variance.

- Start with all data at the root
- Split until nodes are too small or max depth reached
- **Prediction:** mean of observations in the leaf

Think of it as asking: *“What is the single yes/no question that best separates high-sales months from low-sales months?”*

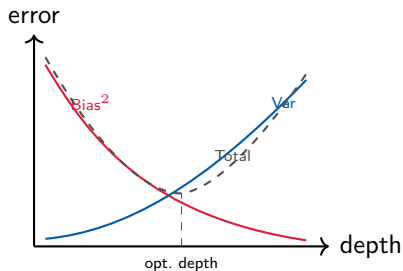
Then repeat within each group.

For classification, replace variance with Gini impurity or entropy (information gain).

## Tree depth and complexity:

- **Depth 1** (stump): one question — high bias, low variance
- **Depth 5–10**: balanced
- **Full tree** (one obs. per leaf): memorizes data — zero bias, extreme variance

**Why trees have high variance:** A single changed training observation can alter the split at the root, changing every prediction below it. Trees are inherently *unstable*.



Tune with `max_depth` and `min_samples_leaf`. Select via `TimeSeriesSplit` cross-validation.

---

## Implementing Decision Trees in Python

One class, a few hyperparameters, interpretable results.

---

```
from sklearn.tree import
DecisionTreeRegressor
from sklearn.model_selection import
TimeSeriesSplit
X = make_lag_features(df, lags=[1,2,3,12])
y = df['sales']

tree = DecisionTreeRegressor(
max_depth=5,
min_samples_leaf=10,
random_state=42)
tree.fit(X_train, y_train)
y_hat = tree.predict(X_test)
```

## Key hyperparameters:

| Parameter        | Effect                  |
|------------------|-------------------------|
| max_depth        | Cap tree depth          |
| min_samples_leaf | Min obs. per leaf       |
| max_features     | Feature subsampling     |
| ccp_alpha        | Cost-complexity pruning |

Always use TimeSeriesSplit for cross-validation. Never use KFold on time series — it leaks future data.

For each feature  $j$ , sum the total reduction in variance across all splits where feature  $j$  was used, weighted by the fraction of observations in each node. Normalize to sum to 1.

### Example: retail sales tree

| Feature                       | Importance |
|-------------------------------|------------|
| Lag 12 (same month last year) | 0.41       |
| Lag 1 (previous month)        | 0.27       |
| Lag 2                         | 0.13       |
| December dummy                | 0.10       |
| Unemployment rate             | 0.09       |

Feature importance tells you **which variables the tree found most useful** for splitting — a quick way to screen which predictors matter most.

Can be biased toward high-cardinality features. SHAP values give more reliable individual-level attributions.

---

One of the main advantages of decision trees is that you can explain *why* a specific prediction was made.

“Our model forecasts \$61,300 in sales for next month because:

1. Last year's same-month sales were above \$38,000 (economy growing)
2. Last month's sales were above \$52,000 (current momentum strong)
3. When both conditions hold, the average outcome in our training data was \$61,300.”

### Visualizing the tree:

`sklearn.tree.plot_tree(tree)` or  
`export_text(tree, feature_names=[...])`

A decision path tells you which conditions led to the forecast. This is useful for business buy-in — managers can validate that the tree is using sensible splits.

---

## Limitations and Preview: Ensembles

A single tree is a weak learner. Combine hundreds to build a strong one.

---

---

## Strengths:

- ✓ Interpretable (can show the path)
- ✓ No feature scaling needed
- ✓ Captures nonlinear relationships
- ✓ Handles missing values well
- ✓ Fast to train

## Weaknesses:

- ✗ High variance — small data changes  $\Rightarrow$  very different tree
- ✗ Flat predictions within each leaf
- ✗ Cannot extrapolate beyond training range
- ✗ Sensitive to outliers in a leaf

**The ensemble solution:** Average many trees trained on different bootstrap samples. This reduces variance dramatically while retaining feature-importance interpretability.

Random Forests improve over single trees by 20–40% in RMSE on standard benchmarks.

---

Linear models cannot capture **interactions** or **threshold effects**. Decision trees can, without needing to specify these relationships in advance.

The **bias–variance tradeoff** means that a too-simple tree underfits (high bias) and a too-deep tree overfits (high variance). Tune with `max_depth` and `min_samples_leaf`.

**CART** greedily minimizes within-region variance at each split. Prediction = mean of observations in the leaf.

Decision trees are **interpretable**: you can trace the exact decision path that led to any prediction.

Single trees have **high variance** — they change dramatically when the data changes slightly. Ensembles (Random Forests, XGBoost) solve this.

*Next: Random Forests — variance reduction through bootstrap aggregation and random feature selection.*

- 
-  Breiman, Leo et al. (1984). *Classification and Regression Trees*. Belmont, CA: Wadsworth.
  -  James, Gareth et al. (2023). *An Introduction to Statistical Learning with Applications in Python*. 1st. New York: Springer. URL: <https://www.statlearning.com/>.
  -  Molnar, Christoph (2022). *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. 2nd. Open access. URL: <https://christophm.github.io/interpretable-ml-book/>.